



프로젝트 최종 발표

멀티센서 기반 유압 시스템 부품 상태 10초 조기판별
및 원인 분석 3D 디지털 트윈

조현재, 홍유나, 박민, 신종건

목차

HydroTwin 프로젝트 발표 구성

01	프로젝트 소개	03 ~ 07
02	데이터 분석 및 AI 모델	08 ~ 12
03	시스템 구현 및 관제	13 ~ 14
04	배포 자동화	15 ~ 16
05	프로젝트 회고. 기대 효과 및 향후 방향	17

팀원 소개 및 역할 분담

TEAM HydroTwin 구성원 및 주요 담당 업무

PM

조현재

PM / Unity

전체 매니징, Unity 3D 디지털 트윈 구축, API 규격 협의 및 시스템 통합

AI

홍유나

AI / UIUX

데이터 분석 및 AI 모델링, SHAP 영향 센서 분석 로직 구현, UI/UX 디자인

BE

박민

Backend / Web

FastAPI 서버 구축, Kafka Producer/Consumer 개발, Web 대시보드 및 실시간 연동

OP

신종건

DevOps / MLOps

Docker/Jenkins CI/CD 파이프라인 구축, pytest 기반 자동화 테스트, MLOps 운영 환경 구성

사용 기술 스택

프로젝트 전반에 적용된 언어, 프레임워크 및 툴 체인

Languages

Python

C#

JavaScript

HTML / CSS

AI & Data

scikit-learn

LightGBM

SHAP

pandas

NumPy

Streaming & Backend

Kafka

FastAPI

Uvicorn

Frontend & 3D

Chart.js

Unity WebGL

DevOps & MLOps

Docker

Jenkins

pytest

Git / GitHub

일정관리 및 프로젝트 산출물

체계적인 일정 관리와 단계별 개발 산출물

관리 산출물

- 프로젝트 기획서
- 마일스톤/역할 분담표
- 요구사항 정의서
- API / JSON Contract 정의서
- 테스트 계획서
- 발표 자료 및 매뉴얼

개발 산출물

- 전처리 파이프라인 (ETL)
- AI 분류 및 회귀 모델 (Scikit-learn)
- SHAP 기반 XAI 분석 모듈
- Kafka 메시지 큐 패키지
- FastAPI 백엔드 서버
- Web Dashboard (React/Tailwind)
- Unity 3D 시각화 모듈
- Docker Compose / Jenkins Pipeline
- pytest 유닛 테스트 코드

Planning

기획 및 분석
요구사항 및 인프라 설계

Development

모델 및 연동 개발
AI 모델링 및 API 구현

Integration

시스템 통합
대시보드 및 Unity 연동

Verification

검증 및 배포
QA 테스트 및 인프라 배포

왜 HydroTwin이 필요한가?

기존 시스템의 한계와 HydroTwin의 개선

As-Is (기존 방식)

- 17개 센서 데이터를 작업자가 개별 확인
- 다변량 물리량 동시 해석의 한계 및 분석 복잡성
- 비숙련자의 이상 부품 탐지 지연
- 이상 발생 후 사후 원인 파악의 비효율성



To-Be

- 10초 Window 기반 4대 핵심 부품 상태 동시 판별
- SHAP 분석 기반 주요 영향 센서 분석 제공
- Web Dashboard와 Unity 3D 직관적 시각화
- 작업자의 선제적 점검 및 대응 지원

HydroTwin 핵심 해결 방식

엔드투엔드 파이프라인 및 4대 핵심 기능



부품 상태 분류 및 조기 탐지

10초 윈도우 활용 신속 분류

다중 설비 스트리밍

Kafka 기반 대용량 데이터 안정 처리

영향 센서 분석

SHAP 모델 활용 판단 기여도 도출

직관적 모니터링

Web Dashboard + Unity 3D 실시간 관제

데이터셋 소개

모델 학습을 위한 유압 시스템 다변량 시계열 데이터셋 소개

데이터셋 개요

- 데이터 규모: **2,205 Cycles** | 사이클당 60초 | 17 Sensors
- 특성: 다변량 및 다중 샘플링

Source : Condition monitoring of hydraulic systems
@UCI Machine Learning Repository

상태 진단 항목 (Targets)

- 1. 냉각기:** 냉각 효율 상태 판별
- 2. 밸브:** 스위칭 지연 상태 판별
- 3. 펌프:** 내부 누수 상태 판별
- 4. 축압기:** 압력 상태 분류

센서 샘플링 속도 (Sampling Rates)

Rate	Sensors
100Hz	PS1-6 (압력), EPS1 (파워)
10Hz	FS1-2 (유량)
1Hz	TS1-4 (온도), VS1 (진동), SE (효율팩터), CE (냉각효율), CP (냉각파워)

* 데이터셋은 유압 테스트 리그의 다양한 동작 상태를 다각도로 수집함

데이터 전처리 및 최종 모델 입력

10초 윈도우 기반의 특징 추출 및 전처리 파이프라인

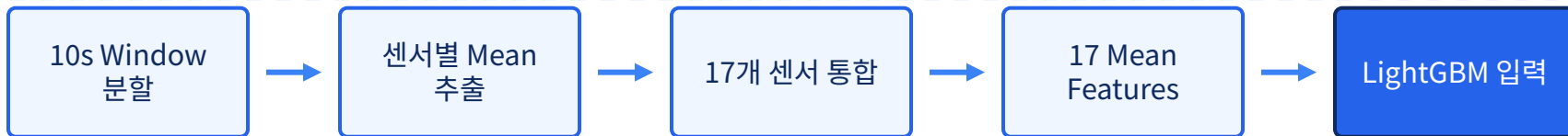
데이터 분할 비율 (Total: 2,205 samples)

Train 70% (1,543)

Val 15% (331)

Test 15% (331)

■ Training Set ■ Validation Set ■ Test Set



* EDA 단계에서는 다양한 통계 특징을 검토했으나 최종 서비스 모델은 17개 Mean Feature를 사용

- 서비스 모델 입력을 17개 Mean Feature로 구성하여 입력 구조 단순화 및 경량화
- EDA 단계 검토 항목: Mean, Std, Min, Max, Range, Slope, RMS (7종)\
- 선정 논리: EDA에서 다양한 특징을 검토했으나, 최종 서비스 모델은 가장 효율적인 17개 Mean Feature만 사용함.
- 학습/검증 데이터 분할

RandomForest vs LightGBM 최종 모델 선정

검증 성능과 추론 속도를 종합한 최종 선정 근거

Model Comparison

구분	LightGBM	RandomForest
Accuracy	0.9909	0.9932(우세)
Macro F1	0.9879	0.9913(우세)

평균 추론 시간	2.87 ms 약 17배 빠름	49.11 ms
----------	---------------------	----------

중간값: LightGBM 2.87 ms RandomForest 46.94 ms
95% 도달 시: LightGBM 3.12 ms RandomForest 65.28 ms

최종 모델 선정 근거

- 01 검증 성능 (정확도)
RF가 Accuracy(0.9932), Macro F1(0.9913)로 소폭 우세
- 02 추론 속도 (초고속)
LGBM(2.87ms)이 RF(49.11ms) 대비 약 17배 빠른 속도 확보
- 03 서비스 적용성 및 확장성
실시간 스트리밍 관제의 초저지연 및 SHAP 분석 연계 최적화

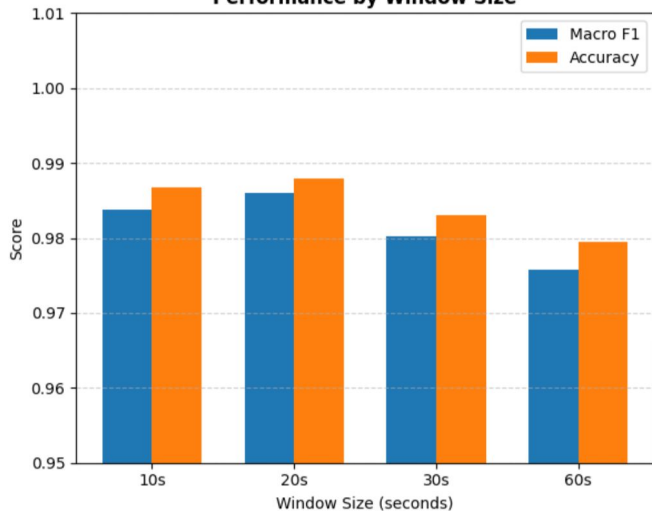
RF 성능 소폭 우세 → LGBM 17배 고속 → ★ 최종 선정: LGBM

정확도와 Macro F1은 RF가 소폭 높았으나, 실제 추론 속도에서 **LightGBM이 약 17배 빠른 응답성**을 보이고 SHAP 기반 원인 분석 연계가 용이하여 최종 서비스 모델로 **LightGBM을 선정**했습니다.

최종 모델 선정 검증 결과

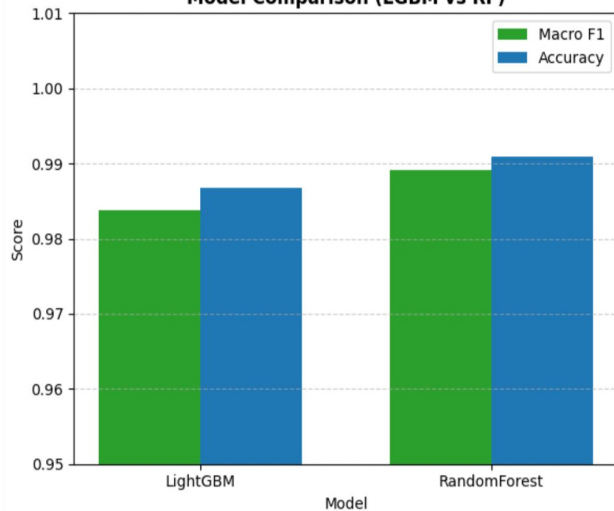
입력 윈도우별 성능과 모델별 정확도·추론 속도 비교

Performance by Window Size



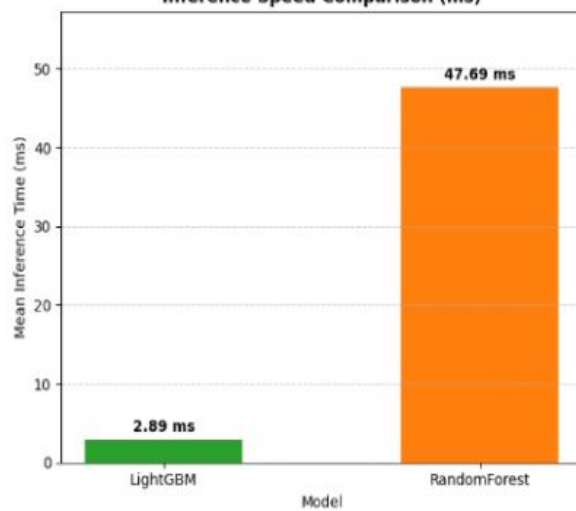
10초 입력에서도 높은 분류 성능 유지
→ 조기 판별 가능

Model Comparison (LGBM vs RF)



RandomForest가 Accuracy·Macro
F1에서 소폭 우세

Inference Speed Comparison (ms)



LightGBM이 RF 대비 약 17배 빠른
추론 속도 확보

SHAP 기반 영향 센서 분석

상태 판별에 영향을 크게 준 주요 센서를 제공하여 작업자의 점검 우선순위 결정 지원

냉각기

- 1st: CP (냉각 성능)
- 2nd: CE (냉각 효율)
- 3rd: EPS1 (소모 전력)

밸브

- 1st: PS2 (압력)
- 2nd: FS1 (유량)
- 3rd: PS3

펌프

- 1st: EPS1
- 2nd: PS1
- 3rd: CP

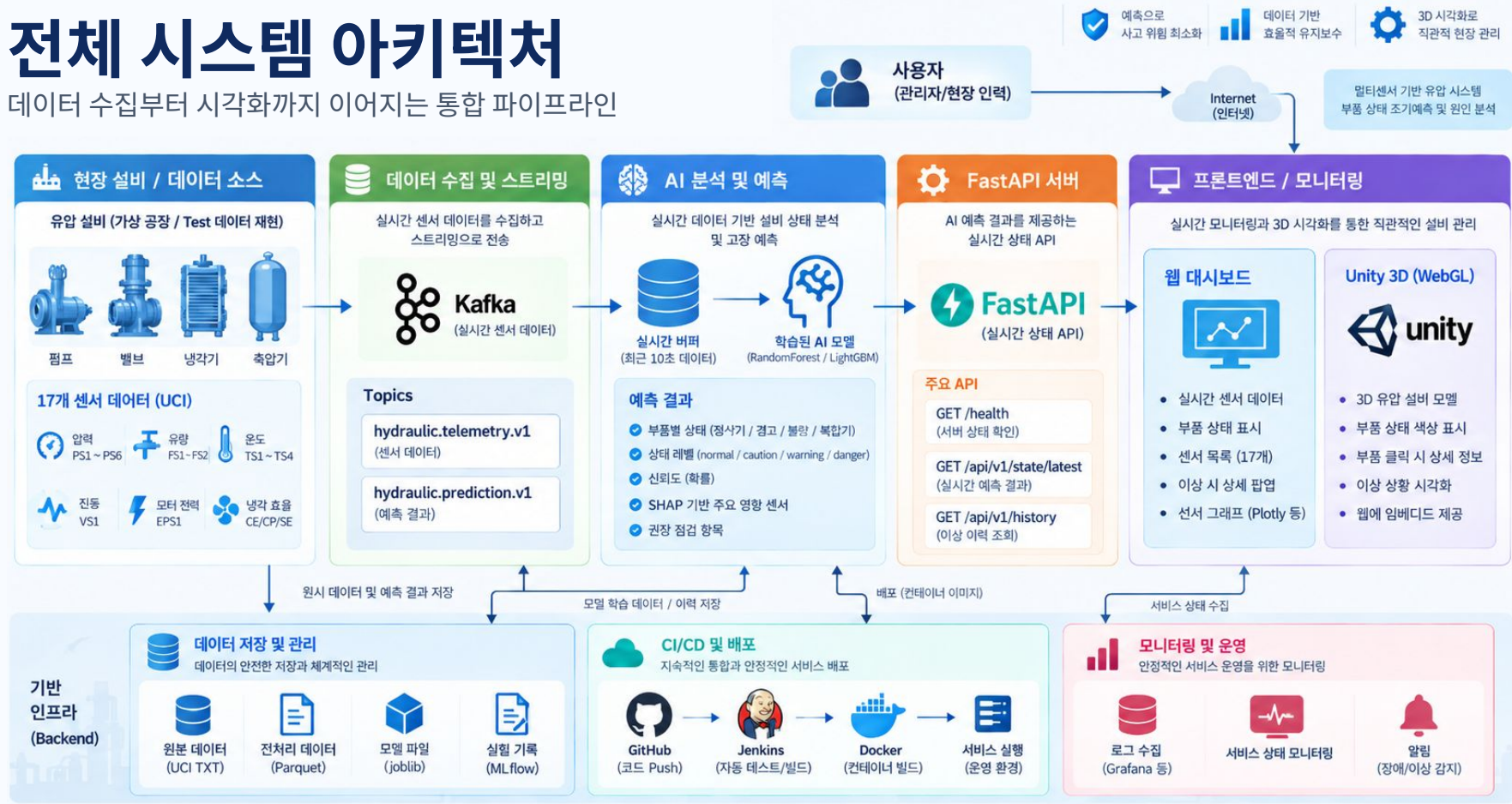
축압기

- 1st: PS3
- 2nd: FS2
- 3rd: PS2

- 본 분석은 물리적 근본 원인을 확정 짓는 것이 아닌, 모델 판단에 기여도가 높은 데이터 패턴을 제시하여 점검 우선순위를 지원함.

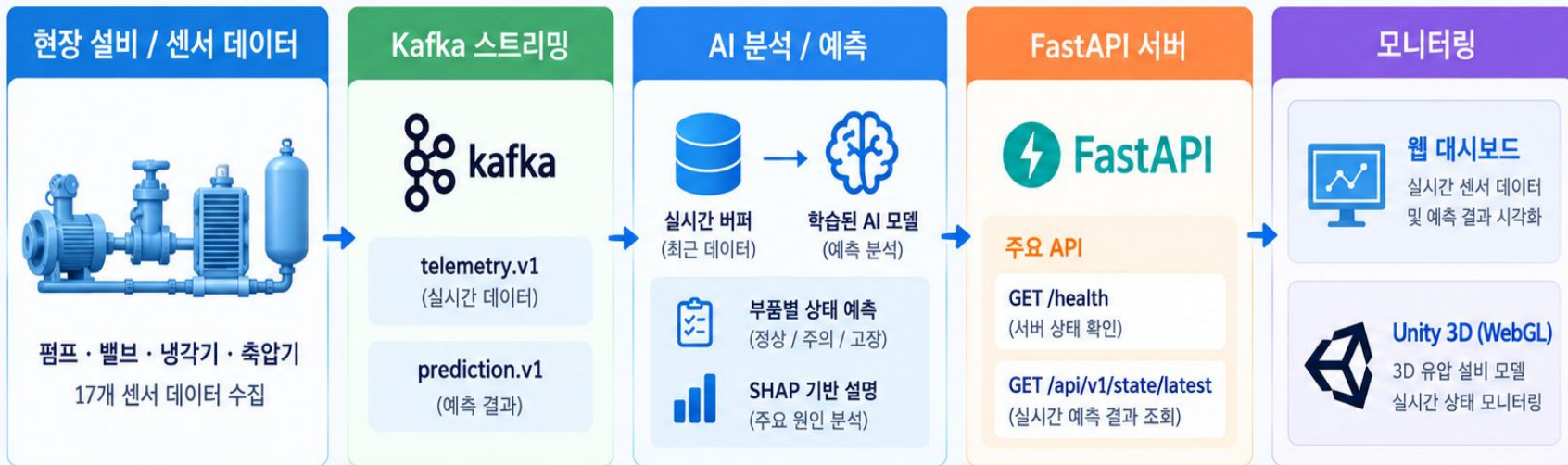
전체 시스템 아키텍처

데이터 수집부터 시각화까지 이어지는 통합 파이프라인



전체 시스템 아키텍처

데이터 수집부터 시각화까지 이어지는 통합 파이프라인



시스템 운영 지원



설비 선택

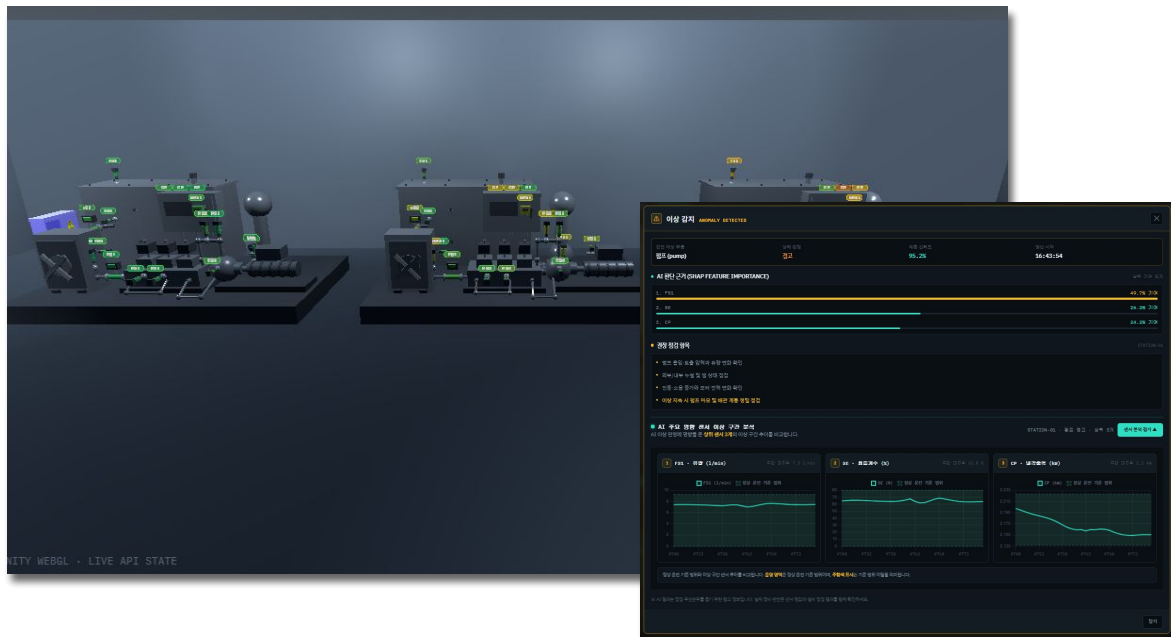
station-01~03
실시간 전환 관제

센서 시계열

17개 센서 1초 주기
실시간 렌더링

Baseline Band

설비별 정상 운전 범위
시각화 제공

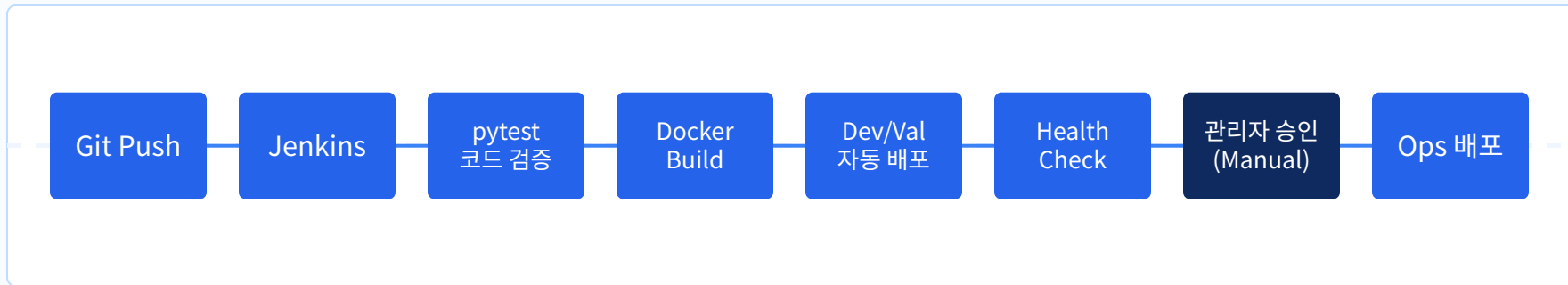


● User-confirmed Alert: 닫기 전까지 팝업 유지

● 데이터 연속성: 캐싱 적용으로 단절 방지

Docker & Jenkins CI/CD 및 MLOps

안정적인 배포와 모델 수명주기 관리를 위한 자동화 파이프라인



Ops Policy

운영 환경 반영은 검증 완료 후 관리자 승인 필수

MLOps 전략

- 드리프트 감지: 데이터 분포 변화 감지
- 선택적 재학습: 성능 저하 시 재학습
- 모델 승격: 성능 게이트 통과 후 승격 관리

프로젝트 트러블슈팅

개발 과정의 문제 해결 사례와 향후 개선을 위한 회고

01. 프로젝트의 당위성..

(ISSUE) 프로젝트 진행중 이 솔루션에 ai가 필수적인가? 라는 근본적인 질문을 받고 고민하게 됨

(SOL) ai가 들어가야하는 부분과 그 이유를 명확하게 정의, 정의한 내용 기반으로 덜어내기/고도화 작업수행

02. 그래프 초기화

(ISSUE) 실시간 렌더링 중 차트 초기화 / **(SOL)** Caching & Sliding Window 적용

03. 알람 자동으로 사라짐

(ISSUE) 자동 알람 소멸로 인지 누락 / **(SOL)** 사용자 수동 확인 적용

기대 효과, 향후 발전 방향

핵심 기대 효과

- 01.** 운영 효율 향상: 10초 조기 판별로 작업자 대응 시간 확보
- 02.** 정비 의사결정 지원: SHAP 영향 센서로 점검 우선순위 판단 지원
- 03.** 직관적 상태 관제: Web/Unity 기반 시각화 및 위치 추적
- 04.** 확장 가능 시스템: Kafka/Docker 기반 다중 설비 확장 구조

향후 발전 방향

- 엣지 디바이스 활용/서버 최적화 및 경량화
- 실제 현장 데이터 교차 검증
- MLOps 기반 모델링 고도화
- Unity 가상 시뮬레이션 환경 완성

감사합니다